“CampusConnect”

an all-in-one platform for campus life

Software Requirements Specification

Prepared by

Student ID	Name
23201336	Nafiz Shahriar Sami
23201518	Jawadul Arham
23241004	Afnan Mohammad Hafiz
23201532	Ibsan Hosaain Ansari
23201393	Eusha Kayenat

1. Introduction **3**
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions, Acronyms, and Abbreviations 3
1.4 References 3
1.5 Overview 4
2. Overall Description **4**
2.1 Product Perspective 4
2.2 Product Features 4
2.3 User Classes and Characteristics 4
2.4 Operating Environment 5
2.5 Constraints 5
2.6 Assumptions and Dependencies 5
3. System Requirements **5**
3.1 Functional Requirements 5
3.1.1 Authentication & Authorization 5
3.1.2 User Profiles, Directory & Attendance 6
3.1.3 Course Registration, Grades & Advising 6
3.1.4 Learning Content & Assignments 6
3.1.5 Real-Time Communication & Notifications .....
3.1.6 Dashboard, Billing & Utilities .....
3.1.7 AI Assistant & Intelligence Services .....
3.2 Non-Functional Requirements 7
3.2.1 Performance Requirements 7
3.2.2 Security Requirements 7
3.2.3 Reliability & Availability 7
3.2.4 Maintainability & Data Integrity 7
3.2.5 Scalability 7
3.3 External Interface Requirements 8
3.3.1 User Interfaces 8
3.3.2 Hardware Interfaces 8
3.3.3 Software Interfaces 8
3.3.4 Communication Interfaces 8
4. Technology Stack & Architectural Overview **8**
4.1 Core Technology Components 8
4.2 MVC Architecture 8
5. Tentative Development Plan (Agile Methodology) **9**
Sprint 1 (Weeks 1–2): Core Setup 9
Sprint 2 (Weeks 3–4): Database Implementation & MVC Architecture 9
Sprint 3 (Weeks 5–6): Admin Authorization and Advising 10
Sprint 4 (Week 7): User Authorization 10

Sprint 5 (Weeks 8–9): Service Quality & Final Touches.....11
6. Acceptance Criteria.....11
7. Conclusion.....11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for CampusConnect, a unified academic portal designed for BRAC University (BRACU). The purpose of this document is to provide a clear and detailed description of the system's intended features, constraints, and behavior to serve as a reference for the development team, stakeholders, and evaluators throughout the project lifecycle.

1.2 Scope

CampusConnect eliminates the need for BRACU students to switch between Connect, buX, and third-party chat apps by uniting academic administration, learning content, and real-time communication into a single portal. The system's scope covers the core modules required to support academic and administrative workflows, along with real-time communication features that replace the need for external chat applications.

The target users are primarily students, but the system also extends functionality to faculty members and admin/registrar staff, ensuring that all key stakeholders in the academic ecosystem are supported within one platform.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS** – Software Requirements Specification
- **BRACU** – BRAC University
- **Connect** – BRACU's existing student portal for academic records
- **buX** – BRACU's existing learning management system (LMS)
- **UI** – User Interface
- **UX** – User Experience

1.4 References

- BRACU Connect and buX documentation

1.5 Overview

This SRS covers the functional and non-functional requirements for the project, the constraints, assumptions, high-level design, and the development plan following Agile methodology. The remainder of this document is organized as follows: subsequent sections detail the overall description of the system, specific functional and non-functional requirements, system design considerations, and the proposed development approach.

2. Overall Description

2.1 Product Perspective

CampusConnect is a standalone academic web application built on a Java Spring Boot backend with a React.js frontend and a PostgreSQL database. It replaces the current fragmented workflow where BRACU students and faculty must separately use Connect (SLMS), buX (LMS), and third-party chat tools. The system integrates with external services where necessary — Google OAuth2 for university SSO and email/push notification gateways for alerts. The application must maintain strict data privacy and access control given the sensitivity of academic records (grades, billing, personal information), in accordance with standard data protection principles for student information systems.

2.2 Product Features

1. **Unified Dashboard:** A single consolidated view of a student's classes, pending assignments, unread messages, and billing status.
2. **Role-Based Access & Dashboards:** Differentiated views and permissions for students, faculty, and admin staff, ensuring each role sees only relevant data and actions.
3. **Course Registration & Grade Tracking:** Real-time seat availability during registration, add/drop functionality, GPA calculation, and transcript viewing.
4. **Learning Content & Assignment Management:** Video lecture streaming with progress tracking, course material repositories, and assignment submission with deadline enforcement.
5. **Real-Time Communication:** Auto-generated chat channels per enrolled course, direct messaging, file sharing, and presence indicators (Discord/Slack-style).
6. **Automated Notifications:** Alerts for new grades, approaching deadlines, class announcements, and billing due dates.
7. **Billing & Payment Status:** A centralized view of tuition and fee payment status, pulled from the university's billing system.
8. **Faculty & Staff Directory:** Searchable directory with contact information and advising details.
9. **Academic Calendar & Global Search:** A unified calendar of academic events alongside a portal-wide search across courses, people, and files.
10. **AI Chatbot Assistant (planned enhancement):** A Codex-powered assistant for answering FAQs, surfacing deadlines, and summarizing course materials — scoped as a future addition beyond the current implementation.

2.3 User Classes and Characteristics

1. **Students (Primary):** The core user base navigating enrollment, coursework, and daily peer-to-peer or peer-to-faculty communication.
2. **Faculty (Secondary):** Academic staff managing course materials, viewing student directories, tracking academic progress, and conducting office hours.

3. **Admin / Registrar Staff (Tertiary):** Back-end personnel overseeing user roles, system permissions, course catalog infrastructure, and billing records.

2.6 Assumptions and Dependencies

- Users have a stable internet connection for telemedicine and streaming.
- Third-party services for payment, SMS/email, and video conferencing are available and reliable.
- Mobile users will grant access to GPS for location-based services when prompted.
- Clinic staff are knowledgeable in using web-based tools and can be trained if needed.

3.1 Functional Requirements

3.1.1 User Management & Authentication

- FR1.1: The system shall allow students, faculty, and staff to log in using their university credentials.
- FR1.2: The system shall support role-based access control (Student, Faculty, Admin).
- FR1.3: The system shall allow users to create and update their profile information.
- FR1.4: The system shall provide a searchable faculty/staff directory.

3.1.2 Course Registration & Academic Records (from Connect)

- FR2.1: The system shall allow students to register/enroll for courses each semester.
- FR2.2: The system shall display real-time grade updates to students.
- FR2.3: The system shall allow faculty to input and update student grades.
- FR2.4: The system shall show billing and payment status for each student.
- FR2.5: The system shall generate academic transcripts/records on request.

3.1.3 Learning Management (from buX)

- FR3.1: The system shall allow faculty to upload course materials (slides, PDFs, documents).
- FR3.2: The system shall support uploading and streaming of video lectures.
- FR3.3: The system shall allow students to submit assignments online before a deadline.
- FR3.4: The system shall allow faculty to review, grade, and provide feedback on submitted assignments.
- FR3.5: The system shall provide a discussion board for each enrolled course.

3.1.4 Communication & Collaboration (from Discord/Slack/Classroom)

- FR4.1: The system shall create a dedicated chat channel for each registered course.
- FR4.2: The system shall allow faculty/admin to post announcements visible to relevant users.
- FR4.3: The system shall send notifications to students/faculty for deadlines, grades, and announcements.
- FR4.4: The system shall support direct messaging (DMs) between users.

- FR4.5: The system shall allow file sharing within chat channels and course discussion boards.
- FR4.6: The system shall allow students to rate faculties based on their performance per course.

3.1.5 Administration

- FR5.1: The system shall allow admins to manage user accounts (create, update, deactivate).
- FR5.2: The system shall allow admins to manage course listings and faculty assignments.
- FR5.3: The system shall allow admins to monitor system usage and generate reports.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- **NFR-1:** The system shall handle multiple concurrent real-time chat sessions across course channels without significant degradation in message delivery speed.
- **NFR-2:** Average page load time shall not exceed 3 seconds under normal network conditions.
- **NFR-3:** Video lecture streaming shall begin playback within 5 seconds of a user request, under normal bandwidth conditions.
- **NFR-4:** Course registration seat-availability checks shall reflect real-time data with no more than a 2-second propagation delay during peak registration periods.

3.2.2 Security Requirements

- **NFR-5:** All communication between client and server shall occur over HTTPS (SSL/TLS).
- **NFR-6:** Sensitive fields in the database (e.g., grades, billing/payment details, personal contact information) shall be encrypted at rest.
- **NFR-7:** Access to academic records (grades, transcripts, billing) shall be restricted by role-based access control, enforced at the API layer, not just the UI.
- **NFR-8:** Authentication shall use university-provided SSO (Google G-Suite/OAuth2) rather than a self-managed credential store, minimizing password-related attack surface.

3.2.3 Reliability & Availability

- **NFR-9:** The system shall maintain 99.5% uptime, excluding scheduled maintenance windows, with priority given to keeping registration and grade-viewing available during add/drop and grading periods.
- **NFR-10:** Data backups (course records, submissions, chat logs) shall be taken daily to minimize data loss in case of failure.
- **NFR-11:** Course materials and previously loaded content shall remain accessible in a degraded read-only mode during brief backend outages.

3.2.4 Maintainability

- **NFR-12:** The codebase shall follow consistent naming conventions and standard Spring Boot layered architecture (Controller–Service–Repository–Entity) to ease future maintenance.

- **NFR-13:** Documentation of REST APIs, data models, and system architecture shall be kept current alongside code changes.
- **NFR-14:** Each core module (auth/permissions, registration, learning content, chat, dashboard) shall be loosely coupled, allowing one module to be modified or extended without breaking others.

3.2.5 Scalability

- **NFR-15:** The system architecture shall accommodate horizontal scaling of the Spring Boot backend and PostgreSQL database with minimal reconfiguration.
- **NFR-16:** The system shall be able to support usage spikes from BRACU's full student body (10,000+ concurrent users) during high-traffic periods such as registration day.

3.2.6 Usability & Accessibility

- **NFR-17:** The interface shall remain usable on low-bandwidth connections, consistent with buX's low-connectivity accessibility goals.
- **NFR-18:** The system shall be responsive across desktop and mobile browsers, since students frequently access course platforms from phones.

4. Technology Stack & Architectural Overview

This section outlines the core technologies selected for the development of CampusConnect and describes the architectural pattern used to structure the system. The chosen stack balances performance, scalability, security, and ease of maintenance, while the MVC architecture ensures a clean separation of concerns across the application.

4.1 Core Technology Components

CampusConnect is built using a modern, modular technology stack, with each component selected for its suitability to the project's academic, real-time, and transactional requirements.

- **Frontend – React (with Vite):** React is used to build a dynamic, component-based user interface, enabling reusable UI elements across student, faculty, and admin dashboards. Vite is used as the build tool and development server, offering fast Hot Module Replacement (HMR) and significantly quicker build times compared to traditional bundlers, improving overall developer productivity.
- **Backend – Spring Boot:** Spring Boot serves as the backend framework, handling business logic, API endpoints, and integration with the database. Its robust ecosystem, built-in security features, and support for RESTful API development make it well-suited for managing the academic, communication, and administrative modules of CampusConnect.
- **Database – PostgreSQL:** PostgreSQL is used as the relational database management system (RDBMS) for storing structured data such as user accounts, course records, grades, and communication logs. Its support for complex queries, data integrity through relational constraints, and scalability make it appropriate for an academic system handling interconnected data.

- **Authentication – JWT (JSON Web Token):** JWT is used to manage secure, stateless authentication across the platform. Upon login, users are issued a signed token that is used to authorize subsequent requests, ensuring that only verified students, faculty, and admin/registrar staff can access role-specific features.
- **Payment Integration – Stripe:** Stripe is integrated to handle secure online payment processing for applicable services within the platform (e.g., fee payments or paid academic services). Stripe's API ensures PCI-compliant, secure handling of transactions without storing sensitive card information directly on CampusConnect's servers.
- **Deployment – Render:** The application is deployed using Render, a cloud platform that supports both frontend and backend hosting. Render provides continuous deployment from version control, simplified environment configuration, and scalable hosting suited for the project's needs.

4.2 MVC Architecture

CampusConnect follows the **Model-View-Controller (MVC)** architectural pattern to ensure a clean separation between data handling, business logic, and user interface, improving maintainability and scalability of the system.

- **Model:** Represents the application's data structure and business logic. In CampusConnect, models correspond to entities such as Students, Faculty, Courses, Messages, and Payments, mapped to PostgreSQL tables via Spring Boot's data layer (JPA/Hibernate). Models are responsible for enforcing data validation and relationships between entities.
- **View:** Represents the presentation layer, implemented using React on the frontend. The View is responsible for rendering data to the user in a clear, interactive format, whether that's a student's dashboard, a faculty course management page, or an admin panel — without containing business logic itself.
- **Controller:** Acts as the intermediary between the Model and View. In Spring Boot, controllers handle incoming HTTP requests from the React frontend, process them (often delegating logic to service classes), interact with the Model layer to retrieve or update data, and return appropriate responses (typically in JSON format) back to the View.

This separation allows each layer to be developed, tested, and maintained independently, supporting the Agile development approach followed throughout the project and making it easier to scale or modify individual components (e.g., adding new modules) without disrupting the overall system.

Tentative Development Plan (Agile Methodology)

Sprint 1 (Weeks 1–2): Project Setup & Authentication

- Set up code repositories, branching rules, and CI/CD pipelines.
- Create the base system architecture and connect module routes (Connect, buX, Chat).
- Build basic user login, logout, and Single Sign-On (SSO).
- Configure team developer environments and shared API standards.

Sprint 2 (Weeks 3–4): Database & MVC Setup

- Model: Create database tables for users, courses, assignments, chats, and billing; write migration scripts for Connect and buX data.
- Controller: Build backend APIs for catalog filtering, directory searches, file uploads, and chat routes.
- View: Create basic UI layouts to display course lists, assignments, academic calendars, and chat channels.

Sprint 3 (Weeks 5–6): User Roles, Advising & Learning Content

- Add role-based permissions for students, faculty, and admins.
- Build the searchable faculty directory, user settings page, and faculty attendance tracker.
- Add course registration with live seat counts, course browsing, and advising workflows.
- Build assignment file submissions with deadline locks, lecture video streaming with progress tracking, and slide uploads.
- Add club recruitment and campus notice boards.

Sprint 4 (Week 7): Chat, Notifications, Dashboard & Payments

- Launch real-time direct messages, course chat rooms, and online/offline status using WebSockets.
- Set up live in-app alerts for grades, due dates, attendance, and notices.
- Build the main dashboard showing classes, assignments, unread chats, and fees.
- Add grade tracking with GPA calculation, semester routine planner, and exam countdowns.
- Integrate Stripe payments for fees and GPS branch locator.

Sprint 5 (Weeks 8–9): AI Tools, Testing & Final Report

- Connect the AI study assistant chatbot and AI semester routine maker.
- Stress-test high-traffic features (registration rush, live chat) and fix low-internet issues.
- Verify data sync and security across merged modules.
- Fix bugs, finalize references, format the document, and submit the final SRS.

6. Conclusion

CampusConnect addresses a real and immediate pain point at BRAC University: students currently navigate three disconnected systems, Connect for registration and grades, buX for course content and

assignments, and separate third-party tools for real-time communication to accomplish what should be a single coherent academic workflow. By unifying these into one portal, CampusConnect reduces context-switching, centralizes critical academic information into a single dashboard, and brings modern real-time communication directly into the academic environment rather than leaving it to external, disconnected apps.